

## Nonparametric Regression

---

The generalized linear model was an extension of the linear model  $y = X\beta + \varepsilon$  to allow the responses  $y$  from the exponential family. The mixed effect models allowed for a much more general treatment of  $\varepsilon$ . We now switch our attention to the linear predictor  $\eta = X\beta$ . We want to make this more flexible. There are a wide variety of available methods, but it is best to start with simple regression. The methods developed here form part of the solution to the multiple predictor problem.

Given fixed  $x_1, \dots, x_n$ , we observe  $y_1, \dots, y_n$  where:

$$y_i = f(x_i) + \varepsilon_i$$

where the  $\varepsilon_i$  are i.i.d. and have mean zero and unknown variance  $\sigma^2$ . The problem is to estimate the function  $f$ .

A parametric approach is to assume that  $f(x)$  belongs to a parametric family of functions:  $f(x|\beta)$ . So  $f$  is known up to a finite number of parameters. Some examples are:

$$\begin{aligned} f(x|\beta) &= \beta_0 + \beta_1 x \\ f(x|\beta) &= \beta_0 + \beta_1 x + \beta_2 x^2 \\ f(x|\beta) &= \beta_0 + \beta_1 x^{\beta_2} \end{aligned}$$

The parametric approach is quite flexible because we are not constrained to just linear predictors as in the first model of the three above. We can add many different types of terms such as polynomials and other functions of the variable to achieve flexible fits. Nonlinear models, such as the third case above, are also parametric in nature. Nevertheless, no matter what finite parametric family we specify, it will always exclude many plausible functions.

The nonparametric approach is to choose  $f$  from some smooth family of functions. Thus the range of potential fits to the data is much larger than the parametric approach. We do need to make some assumptions about  $f$  — that it has some degree of smoothness and continuity, for example, but these restrictions are far less limiting than the parametric way.

The parametric approach has the advantage that it is more efficient if the model is correct. If you have good information about the appropriate model family, you should prefer a parametric model. Parameters may also have intuitive interpretations. Nonparametric models do not have a formulaic way of describing the relationship between the predictors and the response; this often needs to be done graphically. This relates to another advantage of parametric models in that they reduce information

necessary for prediction; you can write down the model formula, typically in a compact form. Nonparametric models are less easily communicated on paper. Parametric models also enable easy utilization of past experience.

The nonparametric approach is more flexible. In modeling new data, one often has very little idea of an appropriate form for the model. We do have a number of heuristic tools using diagnostic plots to help search for this form, but it would be easier to let the modeling approach take care of this search. Another disadvantage of the parametric approach is that one can easily choose the wrong form for the model and this results in bias. The nonparametric approach assumes far less and so is less liable to make bad mistakes. The nonparametric approach is particularly useful when little past experience is available

For our examples we will use three datasets, one real (data on the Old Faithful geyser in Yellowstone National Park, Wyoming, USA) and two simulated, called `exa` and `exb`. The data comes from Härdle (1991). The reason we use simulated data is to see how well the estimates match the true function (which cannot usually be known for real data). We plot the data in the first three panels of Figure 14.1, using a line to mark the true function where known. For `exa`, the true function is  $f(x) = \sin^3(2\pi x^3)$ . For `exb`, it is constant zero, that is,  $f(x) = 0$ :

```
data(exa, package="faraway")
plot(y ~ x, exa, main="Example A")
lines(m ~ x, exa, lwd=2)
data(exb, package="faraway")
plot(y ~ x, exb, main="Example B")
lines(m ~ x, exb, lwd=2)
plot(waiting ~ eruptions, faithful, main="Old Faithful")
```

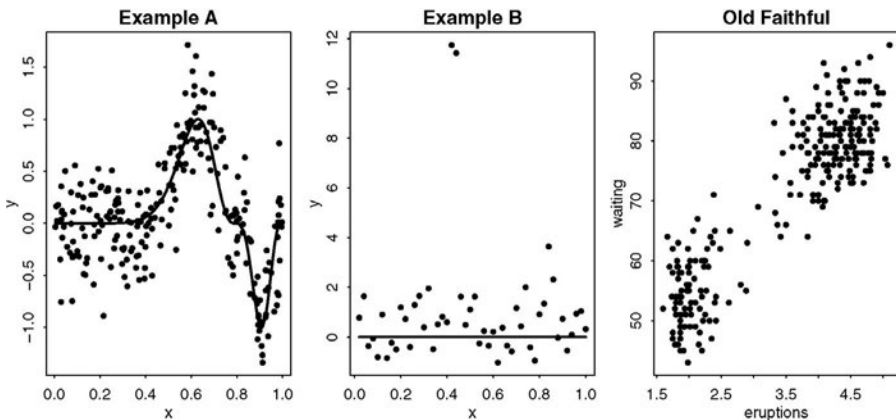


Figure 14.1 Data examples. Example A has varying amounts of curvature, two optima and a point of inflexion. Example B has two outliers. The Old Faithful provides the challenges of real data.

We now examine several widely used nonparametric regression estimators, also known as *smoothers*.

### 14.1 Kernel Estimators

In its simplest form, this is just a moving average estimator. More generally, our estimate of  $f$ , called  $\hat{f}_\lambda(x)$ , is:

$$\hat{f}_\lambda(x) = \frac{1}{n\lambda} \sum_{j=1}^n K\left(\frac{x-x_j}{\lambda}\right) Y_j = \frac{1}{n} \sum_{j=1}^n w_j Y_j \quad \text{where} \quad w_j = K\left(\frac{x-x_j}{\lambda}\right) / \lambda$$

$K$  is a kernel where  $\int K = 1$ . The moving average kernel is rectangular, but smoother kernels can give better results.  $\lambda$  is called the bandwidth, window width or smoothing parameter. It controls the smoothness of the fitted curve.

If the  $x$ s are spaced very unevenly, then this estimator can give poor results. This problem is somewhat ameliorated by the *Nadaraya–Watson* estimator:

$$f_\lambda(x) = \frac{\sum_{j=1}^n w_j Y_j}{\sum_{j=1}^n w_j}$$

We see that this estimator simply modifies the moving average estimator so that it is a true weighted average where the weights for each  $y$  will sum to one.

It is worth understanding the basic asymptotics of kernel estimators. The optimal choice of  $\lambda$  gives:

$$\text{MSE}(x) = E(f(x) - \hat{f}_\lambda(x))^2 = O(n^{-4/5})$$

MSE stands for mean squared error and we see that this decreases at a rate proportional to  $n^{-4/5}$  with the sample size. Compare this to the typical parametric estimator where  $\text{MSE}(x) = O(n^{-1})$ , provided that the parametric model is correct. So the kernel estimator is less efficient. Indeed, the relative difference between the MSEs becomes substantial as the sample size increases. However, if the parametric model is incorrect, the MSE will be  $O(1)$  and the fit will not improve past a certain point even with unlimited data. The advantage of the nonparametric approach is the protection against model specification error. Without assuming much stronger restrictions on  $f$ , nonparametric estimators cannot do better than  $O(n^{-4/5})$ .

The implementation of a kernel estimator requires two choices: the kernel and the smoothing parameter. For the choice of kernel, smoothness and compactness are desirable. We prefer smoothness to ensure that the resulting estimator is smooth, so for example, the uniform kernel will give stepped-looking fit that we may wish to avoid. We also prefer a compact kernel because this ensures that only data, local to the point at which  $f$  is estimated, is used in the fit. This means that the Gaussian kernel is less desirable, because although it is light in the tails, it is not zero, meaning that the contribution of every point to the fit must be computed. The optimal choice under some standard assumptions is the Epanechnikov kernel:

$$K(x) = \begin{cases} \frac{3}{4}(1-x^2) & |x| < 1 \\ 0 & \text{otherwise} \end{cases}$$

This kernel has the advantage of some smoothness, compactness and rapid computation. This latter feature is important for larger datasets, particularly when resampling

techniques like bootstrap are being used. Even so, any sensible choice of kernel will produce acceptable results, so the choice is not crucially important.

The choice of smoothing parameter  $\lambda$  is critical to the performance of the estimator and far more important than the choice of kernel. If the smoothing parameter is too small, the estimator will be too rough; but if it is too large, important features will be smoothed out.

We demonstrate the Nadaraya–Watson estimator next for a variety of choices of bandwidth on the Old Faithful data shown in Figure 14.2. We use the `ksmooth` function which is part of the R base package. This function lacks many useful features that can be found in some other packages, but it is adequate for simple use. The default uses a uniform kernel, which is somewhat rough. We have changed this to the normal kernel:

```
for(bw in c(0.1, 0.5, 2)){
  with(faithful, {
    plot(waiting ~ eruptions, col=gray(0.75))
    lines(ksmooth(eruptions, waiting, "normal", bw))
  })
}
```

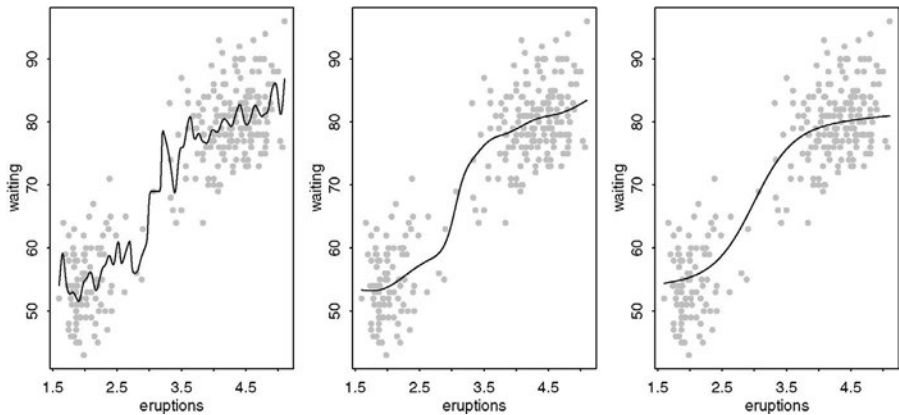


Figure 14.2 *Nadaraya–Watson kernel smoother with a normal kernel for three different bandwidths on the Old Faithful data.*

The central plot in Figure 14.2 is the best choice of the three. Since we do not know the true function relating waiting time and eruption duration, we can only speculate, but it does seem reasonable to expect that this function is quite smooth. The fit on the left does not seem plausible since we would not expect the mean waiting time to vary so much as a function of eruptions. On the other hand, the plot on the right is even smoother than the plot in the middle. It is not so easy to choose between these. Another consideration is that the eye can always visualize additional smoothing, but it is not so easy to imagine what a less smooth fit might look like. For this reason, we recommend picking the least smooth fit that does not show any implausible fluctuations. Of the three plots shown, the middle plot seems best. Smoothers are often used as a graphical aid in interpreting the relationship between variables. In such cases,

visual selection of the amount of smoothing is effective because the user can employ background knowledge to make an appropriate choice and avoid serious mistakes.

You can choose  $\lambda$  interactively using this subjective method. Plot  $\hat{f}_\lambda(x)$  for a range of different  $\lambda$  and pick the one that looks best as we have done above. You may need to iterate the choice of  $\lambda$  to focus your decision. Knowledge about what the true relationship might look like can be readily employed.

In cases where the fitted curve will be used to make numerical predictions of future values, the choice of the amount of smoothing has an immediate effect on the outcome. Even here subjective methods may be used. If this method of selecting the amount of smoothing seems disturbingly subjective, we should also understand that the selection of a family of parametric models for the same data would also involve a great deal of subjective choice although this is often not explicitly recognized. Statistical modeling requires us to use our knowledge of what general forms of relationship might be reasonable. It is not possible to determine these forms from the data in an entirely objective manner. Whichever methodology you use, some subjective decisions will be necessary. It is best to accept this and be honest about what these decisions are.

Even so, automatic methods for selecting the amount of smoothing are also useful. Selecting the amount of smoothing using subjective methods requires time and effort. When a large number of smooths are necessary, some automation is desirable. In other cases, the statistician will want to avoid the explicit appearance of subjectivity in the choice. Cross-validation (CV) is a popular general-purpose method. The criterion is:

$$CV(\lambda) = \frac{1}{n} \sum_{j=1}^n (y_j - \hat{f}_{\lambda(j)}(x_j))^2$$

where  $(j)$  indicates that point  $j$  is left out of the fit. We pick the  $\lambda$  that minimizes this criterion. True cross-validation is computationally expensive, so an approximation to it, known as *generalized cross-validation* or GCV, is sometimes used. There are also many other methods of automatically selecting the  $\lambda$ .

Our practical experience has been that automatic methods, such as CV, often work well, but sometimes produce estimates that are clearly at odds with the amount of smoothing that contextual knowledge would suggest. For this reason, we are unwilling to trust automatic methods completely. We recommend using them as a starting point for a possible interactive exploration of the appropriate amount of smoothing if time permits. They are also useful when very large numbers of smooths are needed such as in the additive modeling approach described in Chapter 15.

When smoothing is used to determine whether  $f$  has certain features such as multiple maximums (called *bump hunting*) or monotonicity, special methods are necessary to choose the amount of smoothing since this choice will determine the outcome of the investigation.

The `sm` library, described in Bowman and Azzalini (1997), allows the computation of the cross-validated choice of smoothing parameter. For example, we find the CV choice of smoothing parameter for the Old Faithful and plot the result:

```
library(sm)
```

```
with(faithful, sm.regression(eruptions, waiting, h=h.select(eruptions,
↪ waiting)))
```

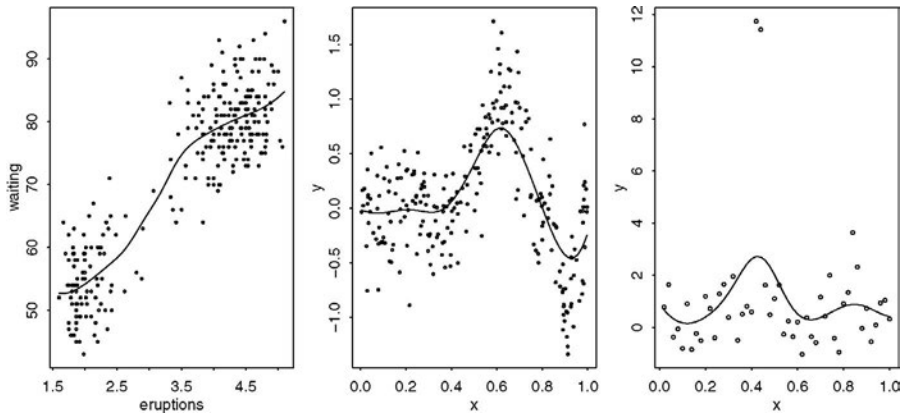


Figure 14.3 The first panel shows the kernel estimated smooth of the Old Faithful data for a cross-validated choice of smoothing parameter. The second and third panels show the resulting fits for Examples A and B, respectively.

We see the resulting fit plotted in the first panel of Figure 14.3. The `sm` package uses a Gaussian kernel where the smoothing parameter is the standard deviation of the kernel.

We repeat the exercise for Example A; the plots are shown in the second panel of Figure 14.3. The resulting fit is somewhat oversmoothed.

```
with(exa, sm.regression(x, y, h=h.select(x,y)))
```

Finally, we compute the fit for Example B as seen in the third panel of Figure 14.3.

```
with(exb, sm.regression(x, y, h=h.select(x,y)))
```

We see that the fitted curve notices the two outliers but does not reach out to them. A much smaller choice of smoothing parameter would allow the fitted curve to pass near these two points but only at the price of a much rougher fit elsewhere.

## 14.2 Splines

**Smoothing Splines:** The model is  $y_i = f(x_i) + \varepsilon_i$ , so in the spirit of least squares, we might choose  $\hat{f}$  to minimize the MSE:  $\frac{1}{n} \sum (y_i - f(x_i))^2$ . The solution is  $\hat{f}(x_i) = y_i$ . This is a “join the dots” regression that is almost certainly too rough. Instead, suppose we choose  $\hat{f}$  to minimize a modified least squares criterion:

$$\frac{1}{n} \sum (Y_i - f(x_i))^2 + \lambda \int [f''(x)]^2 dx$$

where  $\lambda > 0$  is the smoothing parameter and  $\int [f''(x)]^2 dx$  is a *roughness penalty*. When  $f$  is rough, the penalty is large, but when  $f$  is smooth, the penalty is small. Thus the two parts of the criterion balance fit against smoothness. This is the *smoothing spline* fit.

For this choice of roughness penalty, the solution is of a particular form:  $\hat{f}$  is a cubic spline. This means that  $\hat{f}$  is a piecewise cubic polynomial in each interval  $(x_i, x_{i+1})$  (assuming that the  $x_i$ s are unique and sorted). It has the property that  $\hat{f}$ ,  $\hat{f}'$  and  $\hat{f}''$  are continuous. Given that we know the form of the solution, the estimation is reduced to the parametric problem of estimating the coefficients of the polynomials. This can be done in a numerically efficient way.

Several variations on the basic theme are possible. Other choices of roughness penalty can be considered, where penalties on higher-order derivatives lead to fits with more continuous derivatives. We can also use weights by inserting them in the sum of squares part of the criterion. This feature is useful when smoothing splines are means to an end for some larger procedure that requires weighting. A robust version can be developed by modifying the sum of squares criterion to:

$$\sum \rho(y_i - f(x_i)) + \lambda \int [f''(x)]^2 dx$$

where  $\rho(x) = |x|$  is one possible choice.

In R, cross-validation is used to select the smoothing parameter by default. We show this default choice of smoothing for our three test cases:

```
with(faithful, {
  plot(waiting ~ eruptions, col=gray(0.75))
  lines(smooth.spline(eruptions, waiting), lty=2)
})
with(exa, {
  plot(y ~ x, col=gray(0.75))
  lines(x, m)
  lines(smooth.spline(x, y), lty=2)
})
with(exb, {
  plot(y ~ x, col=gray(0.75))
  lines(x, m)
  lines(smooth.spline(x, y), lty=2)
})
```

The fits may be seen in Figure 14.4. The fit for the Old Faithful data looks reasonable. The fit for Example A does a good job of tracking the hills and valleys but overfits in the smoother region. The default choice of smoothing parameter given by CV is a disaster for Example B as the data is just interpolated. This illustrates the danger of blindly relying on automatic smoothing parameter selection methods.

**Regression Splines:** Regression splines differ from smoothing splines in the following way: for regression splines, the number of knots of the B-splines used for the basis are typically much smaller than the sample size. The number of knots chosen controls the amount of smoothing. For smoothing splines, the observed unique  $x$  values are the knots and  $\lambda$  is used to control the smoothing. It is arguable whether the regression spline method is parametric or nonparametric, because once the knots are chosen, a parametric family has been specified with a finite number of parameters. It is the freedom to choose the number of knots that makes the method nonparametric. One of the desirable characteristics of a nonparametric regression estimator is that it should be consistent for smooth functions. This can be achieved for regression

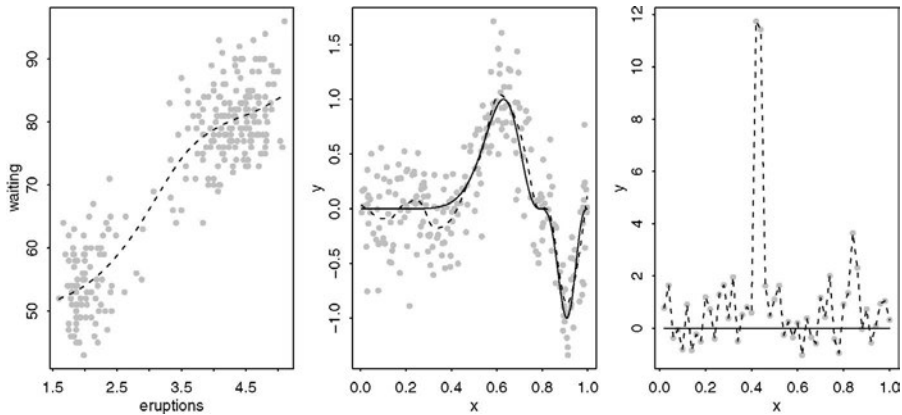


Figure 14.4 *Smoothing spline fits. For Examples A and B, the true function is shown as solid and the spline fit as dashed.*

splines if the number of knots is allowed to increase at an appropriate rate with the sample size.

We demonstrate some regression splines here. We use piecewise linear splines in this example, which are constructed and plotted as follows: knots

```
rhs <- function(x,c) ifelse(x>c,x-c,0)
curve(rhs(x,0.5),0,1)
```

where the spline is shown in the first panel of Figure 14.5. Now we define some knots for Example A:

```
(knots <- 0:9/10)
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
```

and compute a design matrix of splines with knots at these points for each  $x$ :

```
dm <- outer(exa$x,knots,rhs)
matplot(exa$x,dm,type="l",col=1, xlab="x", ylab="")
```

where the basis functions are shown in the second panel of Figure 14.5. Now we compute and display the regression fit:

```
lmod <- lm(exa$y ~ dm)
plot(y ~ x, exa, col=gray(0.75))
lines(exa$x,predict(lmod))
```

where the plot is shown in the first panel of Figure 14.6. Because the basis functions are piecewise linear, the fit is also piecewise linear. A better fit may be obtained by adjusting the knots so that they are denser in regions of greater curvature:

```
newknots <- c(0,0.5,0.6,0.65,0.7,0.75,0.8,0.85,0.9,0.95)
dmn <- outer(exa$x,newknots,rhs)
lmod <- lm(exa$y ~ dmn)
plot(y ~ x, exa, col=gray(0.75))
lines(exa$x,predict(lmod))
```

where the plot is shown in the second panel of Figure 14.6. We obtain a better fit but only by using our knowledge of the true curvature. This knowledge would not be available for real data, so more practical methods place the knots adaptively according to the *estimated* curvature.

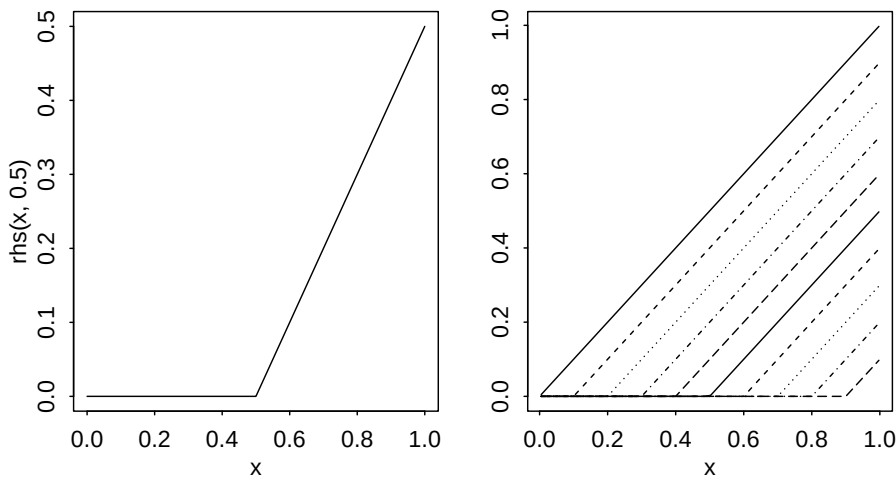


Figure 14.5 One basis function for linear regression splines shown on the left and the complete set shown on the right.

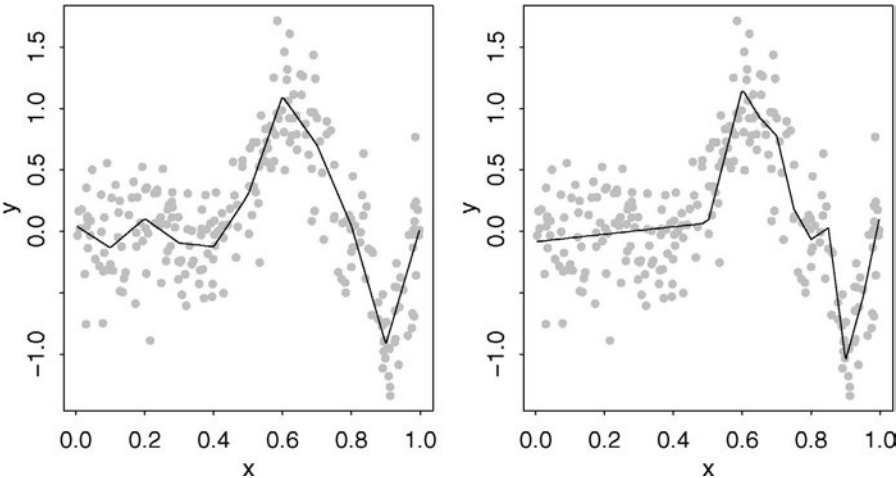


Figure 14.6 Evenly spaced knots fit shown on the left and knots spread relative to the curvature on the right.

One can achieve a smoother fit by using higher-order splines. The `bs()` function can be used to generate the appropriate spline basis. The default is cubic B-splines. We display 12 cubic B-splines evenly spaced on the  $[0,1]$  interval. The splines close to the boundary take a different form as seen in the first panel of Figure 14.7:

```
library(splines)
matplot(bs(seq(0,1,length=1000),df=12),type="l",ylab="",col=1)
```

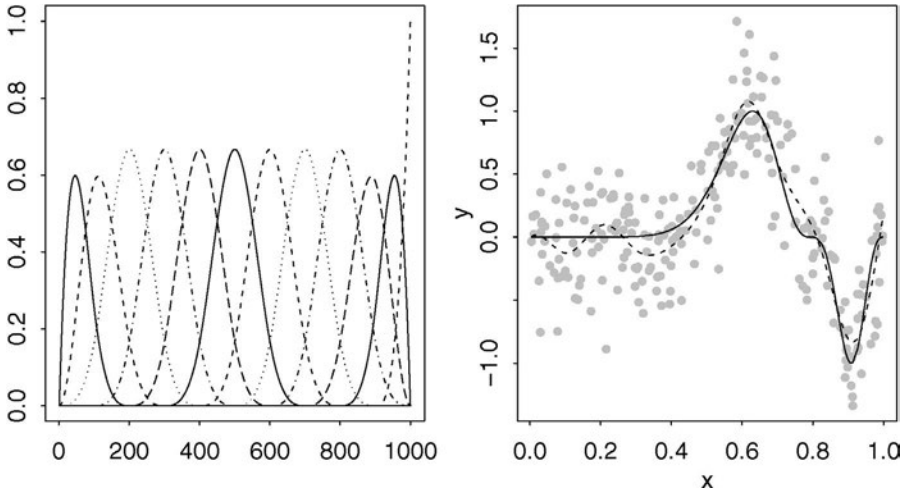


Figure 14.7 A cubic B-spline basis is shown in the left panel and the resulting fit to the Example A data is shown in the right panel.

We can now use least squares to determine the coefficients. We then display the fit as seen in the second panel of Figure 14.7:

```
lmod <- lm(y ~ bs(x,12),exa)
plot(y ~ x, exa, col=gray(0.75))
lines(m ~ x, exa)
lines(predict(lmod) ~ x, exa, lty=2)
```

We see a smooth fit, but again we could do better by placing more knots at the points of high curvature and fewer in the flatter regions.

### 14.3 Local Polynomials

Both the kernel and spline methods have been relatively vulnerable to outliers as seen by their performance on Example B. The fits can be improved with some manual intervention, either to remove the outliers or to increase the smoothing parameters. However, smoothing is frequently just a small part of an analysis and so we might wish to avoid giving each smooth individual attention. Furthermore, habitual removal of outliers is an ad hoc strategy that is better replaced with a method that deals with long-tailed errors gracefully. The local polynomial method combines robustness ideas from linear regression and local fitting ideas from kernel methods.

First we select a window. We then fit a polynomial to the data in that window using robust methods. The predicted response at the middle of the window is the

fitted value. We then slide the window over the range of the data, repeating the fitting process as the window moves. The most well-known implementation of this type of smoothing is called *lowess* or *loess* and is due to Cleveland (1979).

As with any smoothing method, there are choices to be made. We need to choose the order of the polynomial fit. A quadratic allows us to capture peaks and valleys in the function. However, a linear term also performs well and is the default choice in the `loess` function. As with most smoothers, it is important to pick the window width well. The default choice takes three quarters of the data and may not be a good choice as we shall see below.

For the Old Faithful data, the default choice is satisfactory, as seen in the first panel of Figure 14.8:

```
with(faithful, {
  plot(waiting ~ eruptions, col=gray(0.75))
  f <- loess(waiting ~ eruptions)
  i <- order(eruptions)
  lines(f$x[i], f$fitted[i])
})
```

For Example A, the default choice is too large. The choice that minimizes the integrated squared error between the estimated and true function requires a span (proportion of the range) of 0.22. Both fits are seen in the middle panel of Figure 14.8:

```
with(exa, {
  plot(y ~ x, col=gray(0.75))
  lines(m ~ x)
  f <- loess(y ~ x)
  lines(f$x, f$fitted, lty=2)
  f <- loess(y ~ x, span=0.22)
  lines(f$x, f$fitted, lty=5)
})
```

In practice, the true function is, of course, unknown and we would need to select the span ourselves, but this optimal choice does at least show how well loess can do in the best of circumstances. The fit is similar to that for smoothing splines.

For Example B, the optimal choice of span is one (that is all the data). This is not surprising since the true function is a constant and so maximal smoothing is desired. We can see that the robust qualities of loess prevent the fit from becoming too distorted by the two outliers even with the default choice of smoothing span:

```
with(exb, {
  plot(y ~ x, col=gray(0.75))
  lines(m ~ x)
  f <- loess(y ~ x)
  lines(f$x, f$fitted, lty=2)
  f <- loess(y ~ x, span=1)
  lines(f$x, f$fitted, lty=5)
})
```

## 14.4 Confidence Bands

It is helpful to have some expression of uncertainty in the curve estimates. Both the regression spline and loess methods use (local) linear fitting using parametric methods. These same methods naturally generate a standard error which can be used to construct a confidence interval at any point in  $x$ . We may connect these intervals to-

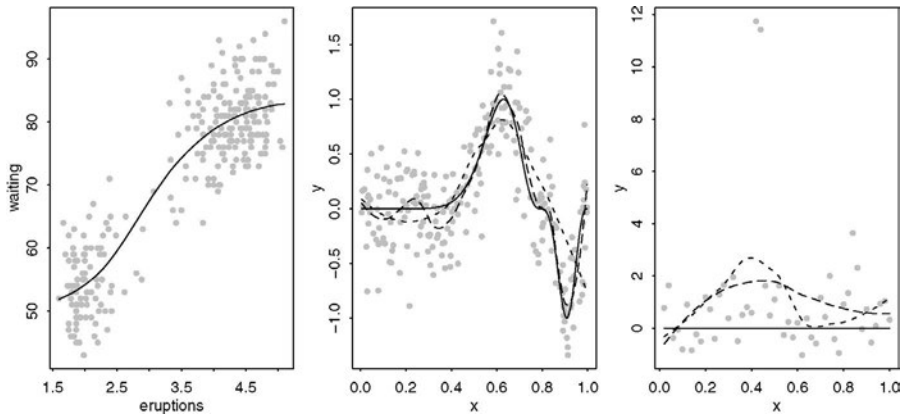


Figure 14.8 *Loess smoothing*: Old Faithful data is shown in the left panel with the default amount of smoothing. Example A data is shown in the middle and B in the right panel. The true function is shown as a solid line along with the default choice (dotted) and respective optimal amounts of smoothing (dashed) are also shown.

gether to form a confidence band. These are conveniently constructed and displayed using the `ggplot2` package.

We construct the 95% confidence band for Example A data using `loess`:

```
ggplot(exa, aes(x=x, y=y)) + geom_point(alpha=0.25) + geom_smooth(
  ↪ method="loess", span=0.22) + geom_line(aes(x=x, y=m), linetype=2)
```

The plot is seen in the first panel of Figure 14.9. We have added the true function as a dashed line. We observe that the true function falls just outside the band in a few areas. However, the band we have constructed is a *pointwise* confidence band. The 95% confidence applies at each point but since we have a wide range of points, the 95% probability of the interval containing the true value cannot hold across the range. For this we would need a *simultaneous* confidence band.

We can also construct a band using splines. We need the `mgcv` package which includes a spline smoother.

```
library(mgcv)
ggplot(exa, aes(x=x, y=y)) + geom_point(alpha=0.25) + geom_smooth(
  ↪ method="gam", formula=y ~ s(x, k=20)) + geom_line(aes(x=x, y=m),
  ↪ linetype=2)
```

We see the resulting plot in the second panel of Figure 14.9. In this case, we have manually chosen the smoothing parameter,  $k=20$ , representing the degrees of freedom in the fit. It is larger to accommodate the variation in this function, producing a better although not perfect fit to that seen in Figure 14.6.

## 14.5 Wavelets

Regression splines are an example of a basis function approach to fitting. We approximate the curve by a family of basis functions,  $\phi_i(x)$ , so that  $\hat{f}(x) = \sum_i c_i \phi_i(x)$ . Thus the fit requires estimating the coefficients,  $c_i$ . The choice of basis functions will

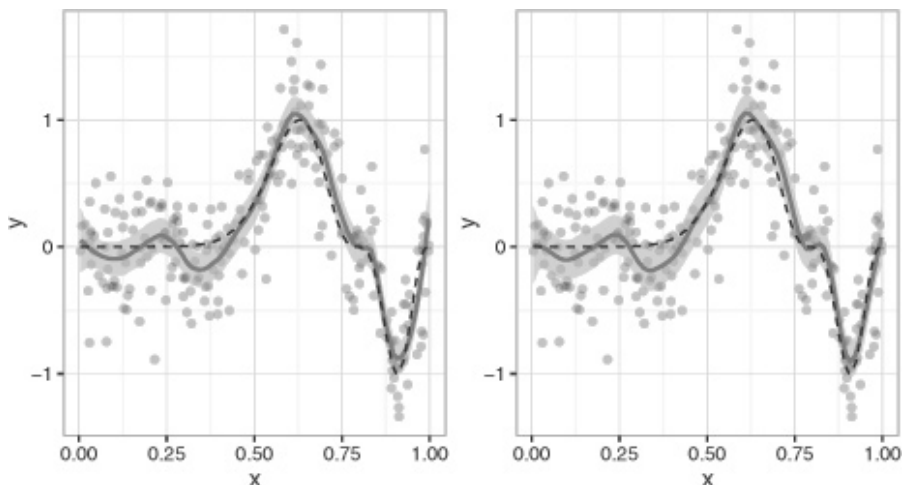


Figure 14.9 95% confidence bands for loess (left) and spline (right) fits to Example A.

determine the properties of the fitted curve. The estimation of  $c_i$  is particularly easy if the basis functions are orthogonal.

Examples of orthogonal bases are orthogonal polynomials and the Fourier basis. The disadvantage of both these families is that the basis functions are not compactly supported so that the fit of each basis function depends on the whole data. This means that these fits lack the desirable local fit properties that we have seen in previously discussed smoothing methods. Although Fourier methods are popular for some applications, particularly those involving periodic data, they are not typically used for general-purpose smoothing.

Cubic B-splines are compactly supported, but they are not orthogonal. *Wavelets* have the advantage that they are compactly supported and can be defined so as to possess the orthogonality property. They also possess the *multiresolution* property which allows them to fit the grosser features of the curve while focusing on the finer detail where necessary.

We begin with the simplest type of wavelet: the *Haar* basis. The *mother wavelet* for the Haar family is defined on the interval  $[0, 1)$  as:

$$w(x) = \begin{cases} 1 & x \leq 1/2 \\ -1 & x > 1/2 \end{cases}$$

We generate the members of the family by dilating and translating this function. The next two members of the family are defined on  $[0, 1/2)$  and  $[1/2, 1)$  by rescaling the mother wavelet to these two intervals. The next four members are defined on the quarter intervals in the same way. We can index the family members by level  $j$  and within the level by  $k$  so that each function will be defined on the interval  $[k/2^j, (k+1)/2^j)$  and takes the form:

$$h_n(x) = 2^{j/2} w(2^j x - k)$$

where  $n = 2^j + k$  and  $0 \leq k \leq 2^j$ . Plotting these functions reveals that they are orthogonal. They are also orthonormal, because they integrate to 1. Furthermore, they have a local basis where the support becomes narrower as the level is increased. Computing the coefficients is particularly quick because of these properties.

Wavelet fitting can be implemented using the `wavethresh` package of Nason (2013). The first step is to make the wavelet decomposition. We will illustrate this with Example A:

```
library(wavethresh)
wds <- wd(exa$y, filter.number=1, family="DaubExPhase")
```

The filter number specifies the complexity of the family. The Haar basis is the simplest available but is not the default choice. We can now show the mother wavelet and wavelet coefficients:

```
draw(wds)
plot(wds)
```

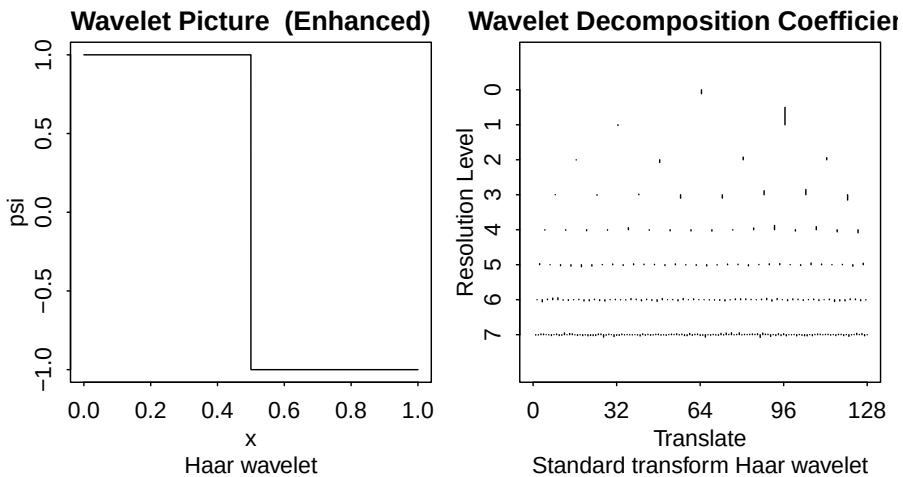


Figure 14.10 *Haar mother wavelet and wavelet coefficients from decomposition for Example A.*

We can see the Haar mother wavelet in the left panel of Figure 14.10. We see the wavelet decomposition in the right panel.

Suppose we wanted to compress the data into a more compact format. Smoothing can be viewed as a form of compression because it retains the important features of the data while discarding some of the finer detail. The smooth is described in terms of the fitted coefficients which are fewer than the number of data points. The method would be called *lossy* since some information about the original data is lost.

For example, suppose we want to smooth the data extensively. We could throw away all the coefficients of level four or higher and then reconstruct the function as follows:

```
wtd <- threshold(wds, policy="manual", value=9999)
fd <- wr(wtd)
```

Only level-three and higher coefficients are retained. There are only  $2^3 = 8$  of these.

The thresholding here applies to level four and higher only by default. Any coefficient less than 9999 in absolute value is set to zero — that is, all of them in this case. The `wr()` inverts the wavelet transform. We now plot the result as seen in the first panel of Figure 14.11:

```
plot(y ~ x, exa, col=gray(0.75))
lines(m ~ x, exa)
lines(fd ~ x, exa, lty=5, lwd=2)
```

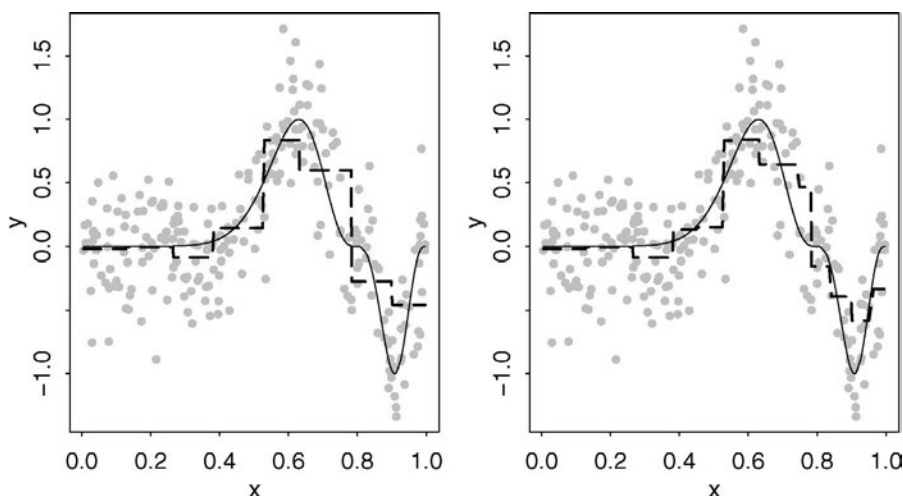


Figure 14.11 *Thresholding and inverting the transform. In the left panel all level-four and above coefficients are zeroed. In the right, the coefficients are thresholded using the default method. The true function is shown as a solid line and the estimate as a dashed line.*

We see that the fit consists of eight constant fits; we expect this since Haar basis is piecewise constant and we have thrown away the higher-order parts leaving just eight coefficients.

Instead of simply throwing away higher-order coefficients, we could zero out only the small coefficients. We choose the threshold using the default method:

```
wtd2 <- threshold(wds)
fd2 <- wr(wtd2)
```

Now we plot the result as seen in the second panel of Figure 14.11.

```
plot(y ~ x, exa, col=gray(0.75))
lines(m ~ x, exa)
lines(fd2 ~ x, exa, lty=5, lwd=2)
```

Again, we see a piecewise constant fit, but now the segments are of varying lengths. Where the function is relatively flat, we do not need the detail from the higher-order terms. Where the function is more variable, the finer detail is helpful.

We could view the thresholded coefficients as a compressed version of the original data (or signal). Some information has been lost in the compression, but the thresholding algorithm ensures that we tend to keep the detail we need, while throwing away noisier elements.

Even so, the fit is not particularly good because the fit is piecewise constant. We

would like to use continuous basis functions while retaining the orthogonality and multiresolution properties. Families of such functions were discovered and described in Daubechies (1991). We illustrate such a form in Figure 14.12 for our data:

```
wds <- wd(exa$y, filter.number=2, bc="interval")
draw(filter.number=2, family="DaubExPhase")
plot(wds)
```

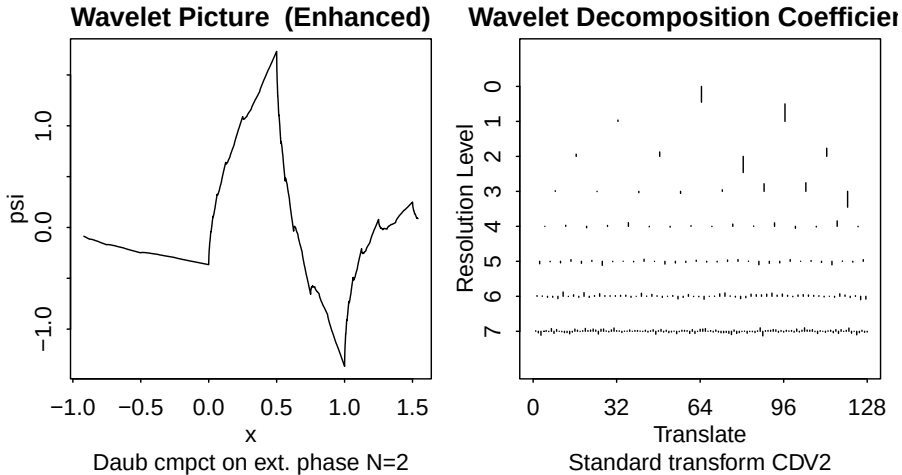


Figure 14.12 *Mother wavelet is shown in the left panel — the Daubechies orthonormal compactly supported wavelet  $N=2$  from the extremal phase family. The right panel shows the wavelet coefficients.*

The mother wavelet takes an unusual form. The function is not explicitly defined, but is implicitly computed from the method for making the wavelet decomposition. Now we try the default thresholding and reconstruct the fit:

```
wtd <- threshold(wds)
fd <- wr(wtd)
plot(y ~ x, exa, col=gray(0.75))
lines(m ~ x, exa)
lines(fd ~ x, exa, lty=2)
```

We can see the fit in Figure 14.13. Although the fit follows the true function quite well, there is still some roughness.

The standard wavelet fitting function is designed only for evenly spaced  $x$  with a number of observations in some power of two. Fortunately, there are 256 observations in Example A, but the Old Faithful set is not so conveniently arranged. The *wavethresh* package implements a method described in Kovac and Silverman (2000) that handles irregularly spaced data. We illustrate how this can be applied here. The method works by interpolating an evenly spaced grid on  $[0, 1]$  using the next highest power of two beyond the size of the data — in this case 512. We need to rescale the predictor to  $[0, 1]$  first for this to work. We have taken the default options throughout except to say that the boundary condition, `bc`, does not assume a periodic function.

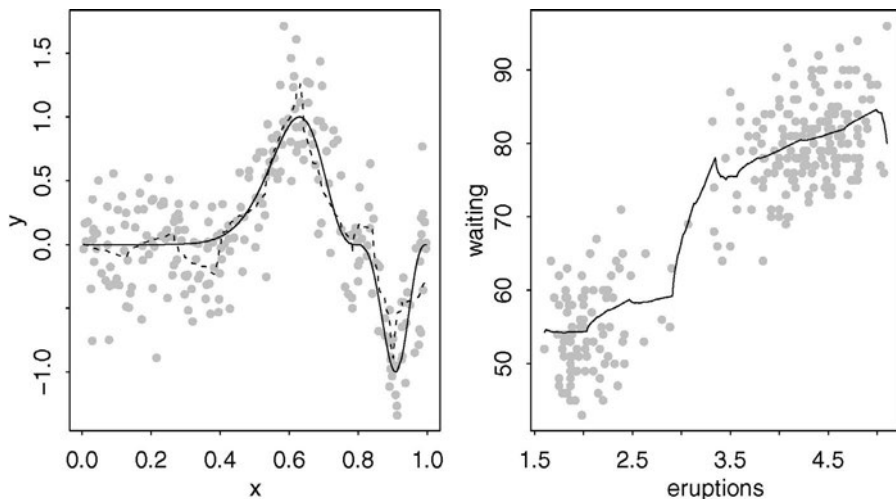


Figure 14.13 *Daubechies wavelet N=2 thresholded fit to the Example A data shown on the left. Irregular wavelet fit to the Old Faithful data is shown on the right.*

```
x <- with(faithful, (eruptions-min(eruptions))/(max(eruptions)-min(
  eruptions)))
gridof <- makegrid(x, faithful$waiting)
wdof <- irregwd(gridof, bc="symmetric")
wtof <- threshold(wdof)
wrof <- wr(wtof)
plot(waiting ~ eruptions, faithful, col=grey(0.75))
with(faithful, lines(seq(min(eruptions),max(eruptions),len=512), wrof))
```

The resulting plot, shown in the second panel of Figure 14.13, reveals a discontinuity in the fit not seen in previous plots. This demonstrates one of the main advantages of the wavelet method in handling and revealing discontinuities. Wavelet methods are particularly useful in processing very large datasets such as those found in image and sound files because the filtering method of thresholding coefficients can drastically reduce file size without losing much information.

## 14.6 Discussion of Methods

We have presented only a selection of the wide variety of methods available. For example, *nearest neighbor* methods adjust for varying density in the predictor space by adjusting window widths to be wider in sparser regions and narrower in denser regions. Window widths are also nonconstant in *variable bandwidth* methods. Such methods are particularly appropriate for functions like Example A where the smoothness of the function varies. We would like to use a smaller bandwidth in regions where the function changes rapidly but a wider one where it is more constant.

Bayesian methods of smoothing are evident in the *Gaussian Process* method as described in Rasmussen and Williams (2006). This method is particularly appropriate if you have prior knowledge and also works well on quite small datasets.

The construction of alternate smoothing methods has long been a popular topic of interest for statisticians and researchers in other fields. Because no definitive solution is possible, this has encouraged the development of a wide range of methods. But it is important not to allow the enthusiasm to solve an interesting technical problem to overshadow the purpose of a data analysis. We propose four possible objectives:

**Description** Sometimes we simply want to draw a line on a scatterplot to aid in the visual interpretation of the relationship displayed. We do not want to spend a lot of time or effort in constructing this smooth. We want a method that is simple and reliable.

**Auxiliary** In some applications, the smoothed fit is needed as part of some larger analysis. The smooth is not the principal aim. For example, the smooth might be used to impute some missing values. In such examples, we need to choose the smoothing method to optimize the wider objective. Sometimes, this means choosing rougher or smoother results than we would pick in a standalone problem.

**Prediction** Interpolating values in noisy data is one example where these methods could be useful. Extrapolation is more problematic as this requires some assumptions about how the function will behave outside the range of the data. Parametric methods do better here as, although extrapolation is inherently risky, it is more transparent how they will behave. We may also want to construct confidence statements which is easier to do with the basis function methods, such as splines, because we can mirror the parametric methods.

**Explanation** In linear modeling, the most common regression question concerns whether there is a relationship between  $x$  and  $y$ . No relationship corresponds to an assertion that the function is constant. We can construct confidence bands for some of the methods. We can then observe whether a constant function fits between the bands to decide the question. Even so, it would be better to directly construct a hypothesis test for which parametric methods are more amenable. Nonparametric methods do at least give us more information about situations where the assertion of no relationship only holds for part of the range of the predictor.

So when should we use nonparametric regression and which particular method should we choose? In the univariate case, we can describe three situations. When there is very little noise, interpolation (or at most, very mild smoothing) is the best way to recover the relation between  $x$  and  $y$ . Splines are good for this purpose. When there is a moderate amount of noise, nonparametric methods are most effective. There is enough noise to make smoothing worthwhile but also enough signal to justify a flexible fit. When the amount of noise becomes larger, parametric methods become relatively more attractive. There is insufficient signal to justify anything more than a simple model.

It is not reasonable to claim that any one smoother is better than the rest. The best choice of smoother will depend on the characteristics of the data and knowledge about the true underlying relationship. The choice will also depend on whether the fit is to be made automatically or with human intervention. When only a single dataset is being considered, it's simple enough to craft the fit and intervene if a particular method produces unreasonable results. If a large number of datasets are to be fit

automatically, then human intervention in each case may not be feasible. In such cases, a reliable and robust smoother may be needed.

We think the loess smoother makes a good all-purpose smoother. It is robust to outliers and yet can produce smooth fits. When you are confident that no outliers are present, smoothing splines is more efficient than local polynomials.

## 14.7 Multivariate Predictors

Given  $\mathbf{x}_1, \dots, \mathbf{x}_n$  where  $\mathbf{x} \in \mathbb{R}^p$ , we observe:

$$y_i = f(\mathbf{x}) + \varepsilon_i \quad i = 1, \dots, n$$

Many of the methods discussed previously extend naturally to higher dimensions, for example, the Nadaraya–Watson estimator becomes:

$$f_{\lambda}(\mathbf{x}) = \frac{\sum_{j=1}^n K\left(\frac{\mathbf{x}-\mathbf{x}_j}{\lambda}\right) Y_j}{\sum_{j=1}^n K\left(\frac{\mathbf{x}-\mathbf{x}_j}{\lambda}\right)}$$

where the kernel  $K$  is typically spherically symmetric, provided the dimensions of  $\mathbf{x}$  are scaled the same way. The spline idea can be used with the introduction of thin plate splines and local polynomials can also be naturally extended.

We can illustrate kernel smoothing in two dimensions. We see in Figure 14.14 that too little smoothing has been used in the first example while the choice for the second example appears about right.

```
data(savings, package="faraway")
y <- savings$sr
x <- cbind(savings$pop15, savings$ddpi)
sm.regression(x, y, h=c(1, 1), xlab="pop15", ylab="growth", zlab="savings
  ↪ rate")
sm.regression(x, y, h=c(5, 5), xlab="pop15", ylab="growth", zlab="savings
  ↪ rate")
```

We can produce a spline surface fit with the help of the `mgcv` package which is explained in detail in the next chapter:

```
library(mgcv)
amod <- gam(sr ~ s(pop15, ddpi), data=savings)
vis.gam(amod, col="gray", ticktype="detailed", theta=-35)
```

We have used the default amount of smoothing and rotated the view of the plot, as seen in the first panel of Figure 14.15, to make it similar to the kernel version. We achieve a smoother but similar fit to the kernel smooth. Two-dimensional smoothing is also possible using `loess`. We need to do more work to construct a 2D grid of predictor values on which we compute the prediction and make the perspective plot:

```
lomod <- loess(sr ~ pop15 + ddpi, data=savings)
xg <- seq(21, 48, len=20)
yg <- seq(0, 17, len=20)
zg <- expand.grid(pop15=xg, ddpi=yg)
persp(xg, yg, predict(lomod, zg), theta=-35, ticktype="detailed", xlab
  ↪ ="pop15", ylab="growth", zlab="savings rate", col="gray")
```

The fit, as seen in the second panel of Figure 14.15, looks similar to the previous smooths but notice the high predicted response for high growth, low `pop15` countries.

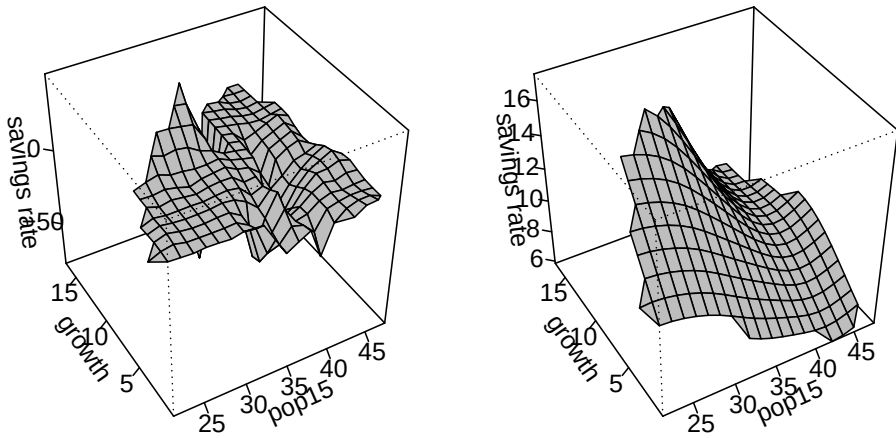


Figure 14.14 *Smoothing savings rate as a function growth and population under 15. Plot on the left is too rough while that on the right seems about right.*

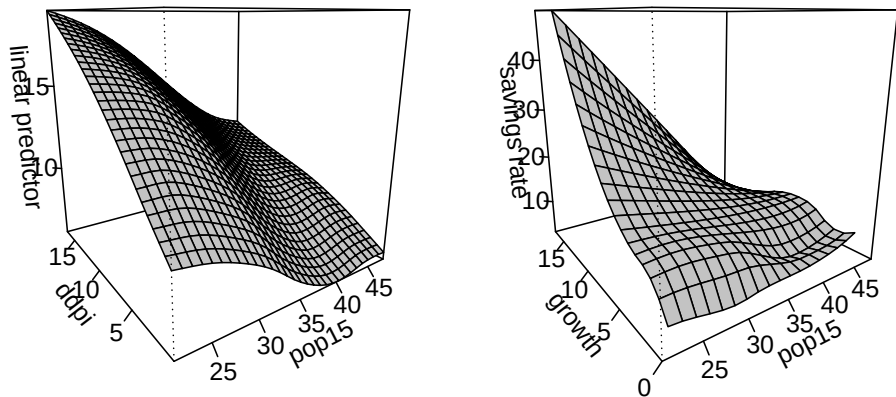


Figure 14.15 *Smoothing spline fit to the savings data shown on the left. Loess smooth is shown on the right.*

Checking the data, we find there are no countries in this region of the predictor space, meaning this represents an extrapolation away from the observed data. The `loess` method uses linear extrapolation, producing the observed result. The kernel-based method does not even attempt to predict outside the range whereas the spline method produced a more restrained prediction. It is difficult to say which is best as we must appeal to subject-matter knowledge to guide our choice.

Developing multivariate estimators is not so difficult but there are problems. Because nonparametric fits are quite complex, we need to visualize them to make sense of them and yet this cannot be done easily for more than two predictors. Most nonparametric regression methods rely on local smoothing; local averaging is the crudest example of this. However, to maintain a stable average we need sufficient points in

the window. For data in high dimensions, the window will need to be wide to capture sufficient points to average. You need an extremely large number of points to cover a high-dimensional space to high density. This is known as the “curse of dimensionality,” a term coined by Bellman (1961). In truth, it should not be called a curse, but rather a blessing, since information on additional variables should have some value, even if it is inconvenient. Our challenge is to make use of this information. Nonparametric regression fits are hard to interpret in higher dimensions where visualization is difficult. Simply extending the one-dimensional method is not effective.

The methods we describe in the following chapters impose additional restrictions on the fitted function to make the problem feasible and the results easier to interpret.

**Further Reading:** For a general review of smoothing methods, see Simonoff (1996). For books on specific methods of smoothing, see Loader (1999), Wahba (1990), Bowman and Azzalini (1997), Wand and Jones (1995) and Eubank (1988). The application of nonparametric regression to the goodness of fit problem may be found in Hart (1997).

## Exercises

1. The dataset `teengamb` concerns a study of teenage gambling in Britain. Take the variables `gamble` as the response and `income` as the predictor.
  - (a) Make a plot of the data.
  - (b) Fit a curve to the data using kernel smoothing with a cross-validated choice of bandwidth. Display the fit on the data. Does the fit look linear?
  - (c) Fit a curve using smoothing splines with the automatically chosen amount of smoothing. Display the fit and report the effective degrees of freedom. Display a fit with somewhat larger degrees of freedom. Was the automatic choice satisfactory?
  - (d) Use loess to fit a curve to the data with the default amount of smoothing. Display the fit.
  - (e) Produce and show a plot with a 95% confidence band for the fit. Is a linear fit plausible?
2. The dataset `uswages` is drawn as a sample from the Current Population Survey in 1988. Predict the `wage` from the years of `education`.
  - (a) Make a plot of the two variables of interest that makes some effort to avoid the problems of overplotting. Repeat the plot but use a log scale for the response.
  - (b) Compute the default smoothing spline fit and display on top of the data. Comment on the quality of the fit.
  - (c) Compute the default lowess fit and display on the fit. Does this method work better than smoothing splines in this instance?
  - (d) For each number of years of education, compute both the mean and the median wage. Construct a plot showing how these means and medians change with education. Which summary works better?

- (e) Instead of means and medians, compute the two quartiles and the median and display on top of the data. (This is a form of quantile regression).
  - (f) Display the lowess fit on the log-transformed data. Do you think it is better to work on the log scale for this data?
3. The dataset `prostate` is from a study of 97 men with prostate cancer who were due to receive a radical prostatectomy. Predict the `lweight` using the `age`.
- (a) Plot the data and comment on the relationship.
  - (b) Fit a curve using kernel methods, plotting the fit on top of the data. What is the effect of the outlier?
  - (c) Compute the smoothing spline fit with the default amount of smoothing. What type of curve has been fit to the data?
  - (d) Fit a loess curve with a 95% confidence band. Do you think a linear fit is plausible for this data?
  - (e) Display all three previous fits on top of the same display and compare.
  - (f) Introduce `lpsa` as a second predictor and show the bivariate fit to the data using smoothing splines.
4. The dataset `divusa` contains data on divorces in the United States from 1920 to 1996. We focus on the `military` variable as it changes by `year`. There really were more military personnel during World War II, so these points are not outliers.
- (a) Plot the number of military personnel and identify the three peaks in the dataset.
  - (b) Fit the data using smoothing splines with the default amount of smoothing, commenting on the quality of the fit. Repeat with lowess, again using the default amount of smoothing.
  - (c) Discuss the utility of smoothing in understanding this data.
5. The `aatemp` data comes from the U.S. Historical Climatology network. They are the annual mean temperatures (in degrees Fahrenheit) in Ann Arbor, Michigan, going back about 150 years.
- (a) Plot the temperature as a function of time and comment on the underlying trend.
  - (b) Fit a least squares line to the data and test whether the slope of the line is different from zero. What is the main drawback of this modeling approach?
  - (c) Fit a Lowess curve to the data using the default amount of smoothing. Display the fit along with a 95% confidence band. What does this say about the underlying trend in the relationship?
  - (d) Fit a regression spline basis to the data with 12 knots. Display the fit on the data.
  - (e) Compare this model to the linear fit using an  $F$ -test. Which model is preferred? What more needs to be explored with spline fit before drawing conclusions?

6. Generate simulated data according to the following model. Use 256 evenly spaced points on  $[0, 1]$  for  $x$ . Let  $y = f(x) + \varepsilon$  with

$$f(x) = \begin{cases} x & x \leq 1/2 \\ x - 1 & x > 1/2 \end{cases}$$

and  $\varepsilon \sim N(0, (0.1)^2)$ .

- (a) Plot the data along with the true function  $f$ .
- (b) Display the Lowess fit on top of the data using the default choice of smoothing. Comment on the quality of the fit. Repeat using a smaller `span` in the smooth.
- (c) Use the default method of wavelet smoothing to compute a fit to the data. Comment on the quality of the fit.
- (d) In the wavelet thresholding, replace the `policy` with `universal`. Does this improve the fit?

This page intentionally left blank